AI Booking Assistant Internal Documentation

1. Architecture

1.1 Tech Stack

Part	Choice
Language and runtime	TypeScript 5 on Node.js 20 or newer
Framework and interface	Next.js 14 (App Router), React 18, Tailwind CSS
Database	PostgreSQL 16 with Drizzle (drizzle-kit for migrations)
Email	Nodemailer for SMTP, Resend as the alternative
AI	one client for every OpenAI-compatible vendor, plus native Google and AWS
Tests and packaging	Vitest (211 tests), Docker and Docker Compose

1.2 Rules I followed

A few standard software principles guided the build. In plain words:

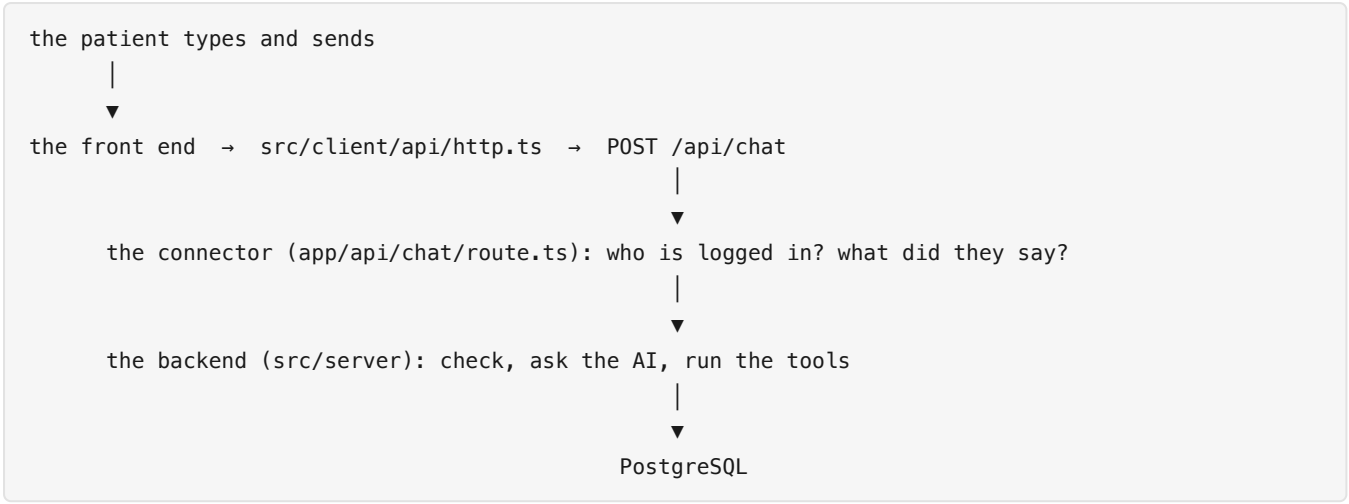
Principle	What it means
Separation of concerns	Each part does one job and does not reach into the others. The booking rules do not touch the AI, the email or the web page
Do not repeat yourself (DRY)	Write a thing once. One place talks to every AI company, one place sends every request from the browser
Keep it simple (KISS)	Pick the plain option. Fewer settings and fewer moving parts means fewer things to get wrong
You are not going to need it (YAGNI)	Do not build for a future nobody asked for. What I left out on purpose is section 6
Depend on a shape, not on a supplier	Talk to a small agreed interface, not a named company. Then swapping the AI or email company is a settings change, not a rewrite (section 1.4)

1.3 The App Structure

The app has three folders, and each one has a single job:

Folder	Name	Job
`src/client`	the front end	What the patient sees. Runs in their browser
`app/api`	the connector	Turns a web address into a function call
`src/server`	the backend	The rules, the AI, the database

A message travels like this:



Three things about this shape:

The connector does no thinking. It checks who is logged in, hands the message to the backend, and returns the answer. Every rule lives behind it.

The front end never touches the database. It only sends web requests, all through one file (`http.ts`), except voice, which has its own client for audio.

Front end and backend are one project. In Next.js a file at `app/api/chat/route.ts` becomes the URL `/api/chat` on its own. So there is one project to deploy, one shared address (no cross-server permissions), and shared TypeScript types that break the build if the two sides ever disagree.

The keys stay on the server: the AI key, the database password and the mail password are only ever read by the backend, never sent to the browser. I checked the built browser files, no key, tool name or database code in any of them.

1.4 Agnostic AI design

The product is not tied to any AI company.

Any provider works. The model sits behind one interface. Most AI companies copy the OpenAI message format, so a single adapter already covers OpenAI, Anthropic, DeepSeek, Gemini and OpenRouter. Each is a row in a table holding its address, its key name and a default model, so adding a company like that is a new row, not new code. `custom` reaches any other compatible company by setting its address.

Two have their own adapter. Amazon Bedrock, because it has no compatible format at all and uses AWS credentials instead of a key. And `gemini-native`, which talks to Google directly; it sits next to the compatible `gemini` row, so Google can be reached either way.

Changing supplier is a settings change:

To do this	Change this
Move the brain from OpenAI to Anthropic	`AI_PROVIDER=anthropic`, and add `ANTHROPIC_API_KEY`
Move email from Gmail to Resend	Remove `SMTP_HOST`, add `RESEND_API_KEY`
Use a different voice company	`VOICE_TTS_PROVIDER`, and its key

No file is edited and the booking rules never move. I only write code for a supplier nobody has connected yet.

Each role can use a different company. The product uses AI for three jobs, and each one chooses separately, because the cheapest good brain and the best voice rarely come from the same company.

Role	Job	Setting
Brain	Picks the tool, writes the reply	`AI_PROVIDER`
Ears	Speech to text	`VOICE_STT_PROVIDER`
Mouth	Text to speech	`VOICE_TTS_PROVIDER`

So a clinic can run a cheap model for the conversation and pay for quality only where the patient actually hears it.

A backup chain. `AI_PROVIDER` also takes a list, tried in order:

```
AI_PROVIDER=openai,anthropic
```

OpenAI answers everything. If OpenAI fails or is too slow, Anthropic answers the same message. The patient waits a little longer and sees no error. A provider with no key is skipped at startup with a warning rather than crashing the app.

1.5 The eight tools

These eight functions are the only things the model can do. It cannot touch the database, the internet or the files directly.

Tool	Takes	Does
`list_services`	none	All treatments, with price and length
`get_professionals_for_service`	service	Dentists who do that treatment
`check_availability`	service, dentist, day	That dentist's free times (past and patient-busy times removed)
`request_login_code`	email	Emails a 6-digit code
`verify_login_code`	email, code	Checks the code, logs the patient in
`get_my_appointments`	none	The logged-in patient's appointments
`create_booking`	service, dentist, time, name	Books it, emails the patient an invite naming both people
`cancel_booking`	booking id	Cancels the patient's own booking

Two details matter. When there are no free times, the tool says why, closed, too late today, patient already booked, or dentist full, so the assistant never says "fully booked" when that is not the reason. And `get_my_appointments` takes no email: it reads the email from the login cookie, so the model cannot ask about another patient. Every value the model sends is checked before the tool runs.

1.6 Who the patient is

There is no login screen. Like a real receptionist, anyone can open the page and ask about treatments, prices, dentists or gaps. The patient only proves who they are at the moment it matters: booking, or viewing their own appointments.

Email code, not a password. A booking ends with a calendar invite (`.ics`), which only works if the email is real. So instead of a password, the patient gets a 6-digit code by email and types it back, one check that proves the address works. A password would only prove they remember a secret.

The code lasts 10 minutes, works once, and is stored only as a hash, so the table is useless if read. After a correct code the browser holds a 7-day token, so booking again the same week asks nothing. The token is also checked against the patient list on every request, not just for a valid signature, otherwise it could outlive a patient the clinic has wiped or restored from an old backup.

Anyone chatting costs money. Without login, anyone can spend the clinic's AI budget, and a per-minute limit alone still allows about 28,800 messages a day. So there are two daily caps:

Who	Cap	Counted by
Not verified	30 messages a day	The chat cookie
Verified patient	100 messages a day	Their email, across all devices

The counts come from the stored messages, not a counter in memory that a restart would reset. Both sit above real use (a booking is 6 to 8 messages), and at the cap the patient is asked to call the clinic, not shown an error.

These caps stop mistakes and casual abuse, not a determined script, an unverified user can clear the cookie for a fresh 30. The real fixes (a clinic-wide cap, a provider spend limit, a human check, an IP limit at a proxy) are in section 6.

1.7 Conversation history

Every message is saved. Only the **last 15** are sent to the AI model.

The window is small on purpose. In a normal chatbot the conversation is the product. Here the booking is the product, and the booking lives in the database.

If a patient asks "when is my appointment?" after 30 messages, the answer comes from a tool reading the database, not from the model remembering. The tools are the memory. The conversation only carries recent wording.

A bigger window would cost more tokens on every single message and give a weak model more text to get lost in.

1.8 Why PostgreSQL and Drizzle

The data is relational: a booking links a patient, a dentist and a treatment, and a dentist only does some treatments. That is rows and links, which is what a relational database is for, so not a document store like MongoDB, which is better when the data has no fixed shape.

Between the relational ones I picked PostgreSQL over MySQL for one reason: Postgres can refuse two overlapping bookings for the same dentist by itself. If two patients tap "book" in the same second, my own code might let both through, but the database checks at the moment it writes, one write at a time, so the second one is rejected. Double booking is the one mistake this product cannot make. The rule is one line, in `drizzle/0001_double_booking_guard.sql`.

Drizzle and migrations. Drizzle lets me write queries in TypeScript, and the types come from the table, so renaming a column and forgetting a query breaks the build, not a booking. Every change to the database shape is a small numbered file, applied in order, so a fresh database and an old one always match.

1.9 Email

Three ways to send email behind one interface, chosen automatically: SMTP if it is configured, otherwise Resend, otherwise the console. So the app runs with no email setup at all.

The demo uses **Gmail SMTP**, because on a free account it is the only one that delivers a booking confirmation to any real patient. Resend only delivers to your own verified address until you verify a domain, and Mailgun's test mode only delivers to a few pre-approved addresses. Gmail sends to anyone, which is what the demo needed to show, but it is a personal mailbox with daily send limits.

For production, Amazon SES is a cheap and reliable choice, well under a dollar at this volume. If the traffic grows, or the clinic wants to send email campaigns as well as confirmations, Mailgun is built for that, with sending domains, delivery reports and reputation tools. Either move is two settings and no code.

---

2. How it decides what to say

The model gets the conversation and a list of the eight tools, but none of the clinic's data. To learn anything, it has to call a tool.

1. Send the conversation and the tool list to the model.
2. If it asks for tools, run them and send back the results.
3. Repeat until it writes a normal reply, or until 8 rounds are used.

The facts come from the code; the model only writes the sentence. Asked "is Dr Chen free on Thursday?", it cannot guess, it calls `check_availability` and reads back the list. The 8-round limit stops a confused model calling tools forever at the clinic's expense.

Two checks keep it honest. Facts are read the moment they are spoken, so nothing is out of date. And every reply is compared with what the tools returned, so a booking the model claims but never made never reaches the patient.

No second AI checks the first. That would be one guessing model checking another, at double the cost. My check is plain code that already knows whether `create_booking` saved a row, a fact beats an opinion, and it is cheaper. A judge model only earns its place offline, grading tone over old conversations before a model switch (section 6), not on every reply.

2.1 Problems I ran into, and how I fixed them

Each time, the lesson was the same: a rule the model keeps breaking belongs in the code, not in a longer prompt.

Problem	Cause	Fix
Asked for **Dr John**, booked **Dr Kate**	The tool took the dentist's id number, and the model sent the wrong one	The tool takes the dentist's **name** now; the server looks up the id
Asked for **9:00**, booked at **2:45pm**	Clinic runs on Kathmandu time (UTC+5:45); "09:00Z" is 14:45 there, and the check only saw open and close hours	Reject any time the clinic does not actually offer, and tell the model to copy a time from the list
Confirmed a booking that never happened	The model wrote "you're booked" with nothing behind it	The reply is checked against the tool results and blocked if unbacked
"Cancel Thursday, book Friday" did only half	A "one question per reply" rule was read as "one thing per message"	Reworded the rule; gave the loop two more rounds
Offered times when the clinic was closed	The model guessing hours	A check in the booking rules
Wrote emojis and code in a spoken reply	The model formatting like a chat app	The output check strips them
Added "that is also 9:00 AM in your local time"	Patient in the clinic's own zone, so it wrote the same time twice	Removed when patient is in the clinic zone, the one I fought three times, which taught the lesson above

---

3. What happens when something fails

Problem	What the system does
One AI provider is down	The next provider in the list answers. A provider with no key is skipped at startup, with a warning
All AI providers are down	The patient reads "I can't reach the booking assistant". Never an error page
A tool gets bad values	Tools never crash. They return an error the model can read and correct. A crash would end the conversation
The model repeats itself forever	The loop stops after 8 rounds and returns a fixed message
The voice key is missing	The microphone turns off and the message names the setting to add. It does not fall back to the robotic voice built into the browser, because that would deliver the rejected thing while looking like it works

---

4. Security

Five layers, and only the first is made of words.

1. The instructions tell the model what it is and what it must not do. This is the weakest layer and I treat it that way.

2. The tools are the only actions. See section 1.5. This is why "ignore your instructions" does not work, there is nothing else for the model to become.

3. Identity comes from the login cookie, never from the model. The request the browser sends contains the message and nothing about who is sending it. The email is read from the signed cookie on the server. There is no field for an attacker to change, so nobody can be talked into booking under another person's address.

4. The database is the final authority. See section 1.8.

5. Fixed rules in the code. See section 2.

Before it runs on a public server

Five things are safe on a laptop and unsafe on the internet:

- The setup file opens the database port to the outside. Remove it.
- The database password is written in the file.
- The login secret falls back to a known development value. On a server it should refuse to start instead.
- There is no HTTPS. A reverse proxy with a free certificate goes in front.
- Voice does not work at all without HTTPS, because browsers block the microphone on insecure pages. So the previous point is not only about security, without it a feature is missing.

What I do not defend against

Picking the right dentist from an unclear sentence depends on the model, and a weak model may pick the wrong one. It cannot pick a dentist who is unqualified or unavailable, because those are refused. And the confirmation says which dentist was booked, so the mistake is visible rather than hidden.

---

5. Cost

To build

Time	22 August to 3 September 2026
Commits	73

Here is a real scenario. The clinic gets **50 patients a day**, every one of them has a full conversation with the assistant, and it runs every day of the month.

The server. I would host it on one **AWS EC2** `t4g.medium` (2 CPU, 4 GB of memory). That is enough for this load with room to spare, and it is the cheapest instance that comfortably runs the app and the database together. The server bill does not move with how busy the clinic is:

AWS EC2, us-east-1, on-demand prices, checked September 2026.

	$ per month
`t4g.medium` (2 CPU, 4 GB memory)	24.53
30 GB disk	2.40
Email (Amazon SES)	under 1
Server total	**about 27**

Email barely registers at this size. The clinic sends a login code and a booking confirmation per patient, so about 3,000 to 4,000 emails a month. The app already supports three real services, so this is a choice, not new code. Prices are as of September 2026 and the two outside AWS change often:

Service	**How it charges**	**This clinic (~3–4k / month)**
Amazon SES	$0.16 per 1,000, no monthly fee	**under $1**
Resend	free up to 3,000 a month, then about $20 a month for 50,000	**$0 to $20**
Mailgun	paid plans from about $15 a month (10,000), about $35 for 50,000	**about $15**

My pick is Amazon SES. The clinic is already on AWS, so SES needs no new account and no new bill, it authenticates with the same AWS credentials the server already has, and at this volume it is under a dollar. Resend is the easiest to wire up and is what I would reach for outside AWS, but its free tier stops at 3,000, just under this clinic's volume.

As the load grows, Mailgun becomes the better one. Its flat plans include the deliverability tooling, sending domains, reputation monitoring, retries and detailed logs, that a clinic sending tens of thousands of confirmations a month actually needs, and at that size the flat fee is cheaper per email than SES's per-message rate. So: SES now for cost and easy integration, Mailgun later if the volume climbs. The demo itself uses Gmail, for the reason in section 1.9.

The AI brain. This moves with use, and it is measured, not guessed: the median message is 8,900 tokens, **94% of it input**, because the instructions and recent history are re-sent every time while the reply is two sentences.

```
50 conversations per day × 30 days  =  1,500 per month
1,500 × 6 messages × 8,900 tokens   ≈  80 million tokens per month
                                    ≈  75M input  +  5M output
```

I ran the demo on **OpenAI** for both the brain and the voice. The brain can be any of these, and the same product costs very differently on each. Prices below are the published per-token rates as of September 2026 and they change often, so treat them as the shape of the answer, not the exact cent:

Brain	Example model	AI per month	With server
Cheap	DeepSeek-V3, or Google Gemini Flash	about $10–15	**about $40**
Cheap (OpenAI)	GPT-4o-mini (what I tested with)	about $15	about $45
Low-middle	Anthropic Claude Haiku	about $80	about $110
Middle (OpenAI)	GPT-4o	about $240	about $270
Middle (Anthropic)	Claude Sonnet	about $300	about $330
Top	Claude Opus	about $1,490	about $1,520

Two takeaways. The server is under 10% of the bill on anything above the cheapest brain, so the real cost choice is one line in the settings file, and correctness does not change down the column, because the model is not what decides. Most of the spend is the 94% of identical text sent every message; OpenAI, the brain I tested, caches that on its own at about half price and the app already sends it unchanged at the front, so the tested setup already saves (turning caching on fully for the other companies is the first item in section 6). **Voice**, if switched on, is a separate bill, per minute of speech in, per character out. I tested it with OpenAI (Whisper to listen, tts-1 to speak); Deepgram and ElevenLabs are the alternative, ElevenLabs giving a more natural voice at a higher price. Rates below are as of September 2026 and change often:

Part	Supplier	Rate	Per voiced conversation
Speech → text	OpenAI Whisper (tested)	about $0.006 / minute	~2 min ≈ $0.012
	Deepgram	about $0.007 / minute	~2 min ≈ $0.015
Text → speech	OpenAI tts-1 (tested)	about $0.015 / 1,000 characters	~900 chars ≈ $0.014
	ElevenLabs	about $0.15–0.20 / 1,000 characters	~900 chars ≈ $0.15

The supplier choice swings this a lot:

Voice stack	Per voiced conversation	All 1,500 a month
OpenAI (what I tested)	about $0.03	**about $40**
Deepgram + ElevenLabs	about $0.17	**about $250–300**

So on the cheaper stack voice adds about as much as a cheap brain; on the ElevenLabs stack it adds as much as a middle brain and can double the whole bill. That is why voice is a setting the clinic turns on, not something always running. ElevenLabs in particular bills by subscription credits rather than pure usage, so its real number depends on the plan; the figure here is the pay-as-you-go shape.

To maintain

The suppliers change more often than the code does. A model gets renamed, a free plan starts refusing a voice it used to allow, a company changes what it expects. I handle all of it the same way: read the error the company sends back and react to that, instead of keeping my own list of model names and settings that goes stale the day I write it.

Normal maintenance is updating libraries and checking prices. Nothing runs in the background that needs watching, and the database is the only thing holding data, so backups are the only real operations work.

---

6. Risks and missing features

These were decisions, not mistakes.

Not built	Reason
Prompt caching, in full	94% of the cost is identical text every message. OpenAI (what I tested) caches it automatically, so the tested setup already saves; the small per-company marker for Anthropic and Gemini is left because it differs per company. Doing it everywhere is roughly $140 a month instead of $300 on a middle model
A clinic-wide daily cap	The caps in section 1.6 are per caller, and an unverified user can clear their cookie to reset. A cap on the clinic's whole day is the one nobody can reset, it turns a runaway bill into a bounded outage
A shared limit counter (Redis)	The per-minute limit is kept in each server's memory and trusts a value the browser can send. Redis, a fast store shared by all the servers, would hold the counts in one place the browser cannot touch. Fine behind one proxy today; Redis is for more than one server
Changing an appointment	Today the patient cancels and rebooks. A real change would keep the calendar id so the existing invite updates instead of being replaced
Interrupting the voice	Talking over the assistant needs two-way streaming, a different design. Instead, the turn ends after 1.6 seconds of silence
Calendar accounts	The invite works with no accounts. Auto-acceptance would need every dentist to connect a calendar and the clinic to keep those credentials
A screen for clinic data	Dentists, treatments and prices live in a code file; a clinic cannot change them without a developer
Model quality testing	Nothing measures how often the model picks the right dentist from a vague sentence, worth checking before switching model in production